

Ciclo II

Desarrollo de Aplicaciones Web

Sesión 2: Operadores y Estructuras Repetitivas

Mg. Dennis Antonio Pérez Escobar

Agenda de la Sesión

- Recapitulación: Sesión 1
- 2.1. Operadores en JavaScript
 - Operadores Aritméticos y de Asignación
 - Lógicos, de Comparación y Ternario
- 2.2. Estructuras de Control (Repetitivas)
 - El principio DRY (Don't Repeat Yourself)
 - Ciclos `while` y `do...while`
 - El bucle `for` y su aplicación en el DOM

Recapitulando: Sesión Anterior

Variables y Mutabilidad

Descartamos `var`. Utilizamos `const` como regla general para proteger la memoria, y `let` únicamente cuando el algoritmo exige reasignación.

Tipos Primitivos vs DOM

JavaScript maneja tipos dinámicos (String, Number, Boolean). Es vital identificar el tipo correcto antes de inyectar datos en la interfaz.

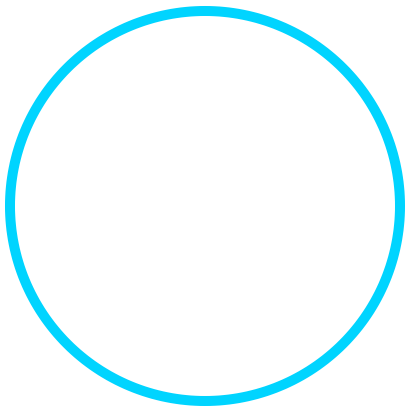
Toma de Decisiones

Las sentencias `if/else` bifurcan el código. El `switch` organiza múltiples casos específicos (requiere `break`).

Evaluación en Cortocircuito

Técnica ágil usando `||` (para valores por defecto) y `&&` (para proteger ejecuciones de funciones).

Conociendo al Docente



Mg. Dennis Pérez

- Máster en Arquitectura de Software e Ing. en Ciencias de la Computación (UDB).
- CEO y Fundador de Tecnova Sv ("Soluciones Tecnológicas Empresariales").
- Especialista en desarrollo corporativo moderno (.NET, Angular, JS nativo).
- Enfoque en código limpio, escalabilidad y arquitecturas sin dependencias innecesarias.

2.1. Operadores de JavaScript

Si las variables son los **sustantivos** de nuestro código (los datos), los operadores son los **verbos**. Son los símbolos que le indican al motor V8 cómo manipular, comparar o asignar esos datos.

Aritméticos

Matemática básica y modular.

Asignación

Guardar y actualizar valores en memoria.

Comparación

Evaluar relaciones (mayor, igual, distinto).

Lógicos

Combinar expresiones booleanas (AND, OR).

2.1. Aritméticos y Asignación

Aritméticos

Además de los clásicos (+ , - , * , /), destacan:

- **Módulo (%)**: Devuelve el residuo de una división. Útil para saber si un número es par.
- **Incremento (++) / Decremento (--)**: Suma o resta 1 a la variable.

```
let contador = 5;
contador++; // Ahora vale 6
let esPar = (10 % 2) === 0; // true
```

Asignación Compuesta

Sintaxis resumida para actualizar variables iterativas sin repetir su nombre.

```
// Tradicional
totalCarrito = totalCarrito + precioItem;

// Asignación Compuesta (Profesional)
totalCarrito += precioItem;

// También aplica para cadenas:
htmlOutput += "<div>Nuevo item</div>";
```

2.1. Comparación: El peligro de la Coerción

En JavaScript, evaluar igualdades tiene una trampa histórica relacionada con los tipos dinámicos. En la industria, **siempre utilizamos la Igualdad Estricta**.

Igualdad Débil (==) - EVITAR

Compara el valor, pero si los tipos son distintos, JS intenta convertirlos de fondo (Coerción). Genera bugs silenciosos.

```
"5" == 5    // true 🚨
0 == false  // true 🚨
"" == 0     // true 🚨
```

Igualdad Estricta (===) - ESTÁNDAR

Compara el valor **y el tipo de dato** explícitamente. Es la regla de oro para evitar vulnerabilidades lógicas.

```
"5" === 5    // false ✅
0 === false  // false ✅
"" === 0     // false ✅
```

2.1. El Operador Ternario

Es la única estructura de JS que requiere tres operandos. Actúa como un bloque `if/else` condensado, ideal para asignar valores basados en una condición directamente.

Sintaxis: `condición ? valor_si_true : valor_si_false;`

La forma antigua (4 líneas):

```
let estado;
if (edad >= 18) {
  estado = "Mayor";
} else {
  estado = "Menor";
}
```

La forma moderna y limpia (1 línea):

```
const estado = (edad >= 18) ? "Mayor" : "Menor";
```

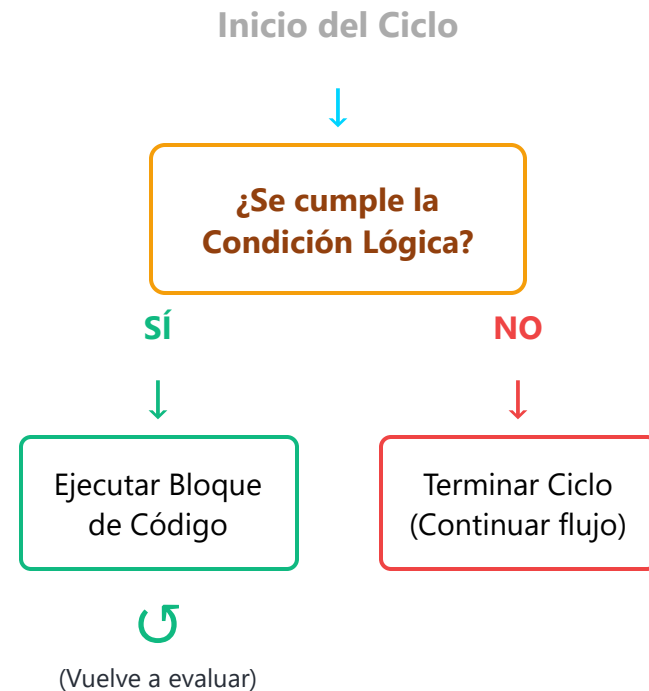
Nota: Esto permite usar `const` porque la asignación se hace en una sola evaluación continua.

2.2. Estructuras Repetitivas (Ciclos)

El Principio D.R.Y.

Don't Repeat Yourself. Si copias y pegas el mismo bloque de código 10 veces para procesar 10 elementos, tu arquitectura es insostenible.

Los ciclos automatizan tareas repetitivas iterando sobre colecciones de datos o ejecutándose hasta que una condición deje de cumplirse.



2.2. Ciclos:

`while`

y

`do...while`

`while`

(Mientras)

Evalúa la condición **antes** de ejecutar el bloque. Si es falsa desde el inicio, el código interno jamás se ejecuta. Útil para "escuchar" eventos.

```
let intentos = 0;
while (intentos < 3) {
  console.log("Procesando login...");
  intentos++; // ¡Crucial para evitar bucle infinito!
}
```

`do...while`

Garantiza que el bloque se ejecute **al menos una vez**, ya que evalúa la condición al final. Ideal para menús interactivos o peticiones iniciales.

```
let sesionIniciada = false;
do {
  mostrarFormularioLogin();
  // Lógica que podría cambiar la variable a true
} while (!sesionIniciada);
```

2.2. El Clásico Bucle

for

Es la estructura más utilizada cuando **sabemos exactamente cuántas veces** queremos iterar (por ejemplo, el tamaño de un array de datos JSON).

```
for (inicialización ; condición ; actualización) { ... }
```

```
for (let i = 0; i < 5; i++) {  
  console.log("Generando fila HTML número: " + i);  
}
```

- **let i = 0** : Crea la variable contadora local (se ejecuta una sola vez).
- **i < 5** : Evalúa antes de cada iteración. Si es true, entra.
- **i++** : Incrementa el contador al finalizar el bloque interno.

2.2. Aplicación Industrial de los Ciclos

En el desarrollo de aplicaciones web reales sin frameworks (Vanilla JS), los ciclos son el motor para renderizar interfaces dinámicas.



Tablas Dinámicas y Cuadrículas

Iterar un JSON devuelto por una API para construir filas `<tr>` y columnas `<td>` e inyectarlas al DOM.



Procesamiento de Carrito

Recorrer el array de productos seleccionados para calcular el total matemático mediante operadores aritméticos y de asignación.



Polling de APIs

Usar ciclos controlados por tiempo para verificar si un servidor externo terminó de procesar un archivo (ej. facturación electrónica).



Limpieza del DOM

Iterar sobre colecciones de nodos HTML para remover elementos u ocultarlos al aplicar filtros de búsqueda interactivos.

Modelado Mental: Ciclos + Operadores

Uniendo todo: Usamos operadores para procesar los datos y un ciclo para generar la vista.

```
// Supongamos un array de 3 productos
let totalPagar = 0;
let listaHTML = "";

for (let i = 0; i < carrito.length; i++) {
  // Asignación compuesta + matemática
  totalPagar += carrito[i].precio;

  // Concatenación del DOM
  listaHTML += "<li>" + carrito[i].nombre + "</li>";
}

document.getElementById('caja').innerHTML = listaHTML;
```

Flujo de Ejecución (La Consola)

Iteración 0: total=25, lista="Mouse"

Iteración 1: total=125, lista="...Teclado"

Iteración 2: total=425, lista="...Monitor"

► **Ciclo termina (i=3). Se inyecta al DOM.**

¿Preguntas?

Siguiente Paso: La Práctica

Preparar el IDE para interactuar con los ejercicios de ciclos.

